

Predicting FIFA Player Overall Rating

An End-to-End MLOps Pipeline on the European Soccer Database

Group Members

Name	Student No.
Martim Líbano Monteiro	20250429
Simão José	20250410
Francisco Henriques	20250417
Pedro Carrasqueira	20250488
António Parafita	20250363

1 Problem, data and success metrics

This project builds an MLOps pipeline to predict a football player’s FIFA `overall_rating` based on their in-game attributes. The dataset comes from Kaggle’s European Soccer Database, a SQLite dump covering 11 European leagues from 2008 to 2016. We mainly rely on the `Player_Attributes` table, which provides about 184,000 player snapshots containing 33 skill ratings (on a 0-100 scale), preferred foot, and work-rate stats. We also joined the `Player` table to add height, weight, and birth date.

We chose this dataset because it includes common data engineering issues: missing values, inconsistent categorical fields, date parsing, and a leakage-prone feature that is too close to the target. After cleaning, the valid `overall_rating` values range from 42 to 90. To keep development cycles fast, we run the pipeline on a fixed 6,000-row sample (seed 42) so a full local run completes in a few minutes. The full table can be built on demand with `scripts/build_dataset.py`.

Since the target ratings are bounded and relatively symmetric (mean around 68), we model them directly without any log transformations. We set our baseline success metrics before training, which are evaluated on a held-out test set in Table 1.

Metric	Role	Target (PoC)	Achieved
RMSE (pts)	Primary; penalises large errors	≤ 3.0	1.45
MAE (pts)	Typical absolute error	≤ 2.5	1.05
R^2	Share of rating variance explained	≥ 0.85	0.954
MAPE (%)	Relative error for stakeholders	≤ 5	1.57
Within 2 pts (%)	How often a prediction is close	≥ 70	86.8

Table 1: Success metrics, targets and results. The champion (HistGradientBoosting) meets every criterion on the held-out test set.

2 Project planning and pipeline architecture

The project is implemented as separate Kedro pipelines rather than a single notebook, which makes each stage runnable and testable on its own. The pipeline covers data quality checks, cleaning, feature generation, model training with MLflow tracking, explainability, containerized serving, and drift monitoring. Each part can run independently through Kedro.

We structured the data using Kedro’s standard layering (`01_raw` to `08_reporting`). As shown in Figure 1, the architecture consists of eight interconnected pipelines. The data prep stages run sequentially, followed by two parallel branches that use the trained champion model. Running `kedro run` executes the training DAG through model training. Downstream prediction and drift checks run as named pipelines, so we can trigger individual components with `kedro run --pipeline <name>`. This structure keeps retraining, batch scoring, and drift checks as separate operational tasks: `model_predict` can score new data, and `data_drifts` can check for distribution shifts without needing to retrain the model from scratch.

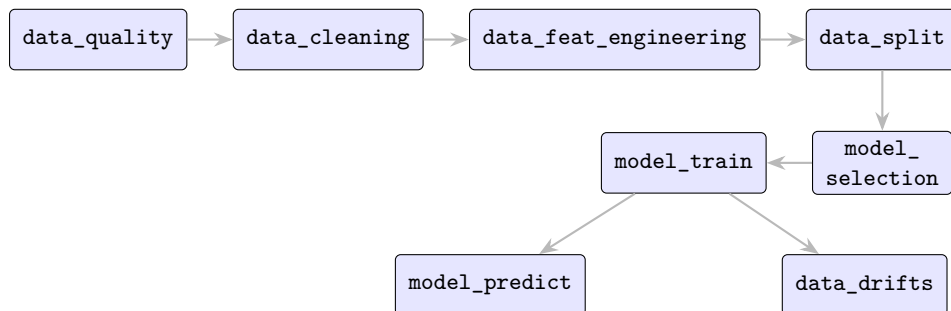


Figure 1: Pipeline DAG. The upper row prepares data, `model_selection` and `model_train` produce the champion, and `model_predict` and `data_drifts` consume it downstream.

A single `modeling.py` module holds the full preprocessing and estimator chain, so the model compared during cross-validation is the same object that is trained and served. The serving layer calls the same `preprocess_for_inference` routine as the offline pipeline, which reduces the risk of training-serving skew. Saved medians cover the skill attributes stored in the serving metadata, but the API should not be described as accepting arbitrary omitted request fields. A row-level parity test would make this check explicit by verifying that both paths produce identical output on the same input.

3 Results and conclusions

3.1 Data exploration and cleaning

Initial EDA (`exploratory_data_analysis.ipynb`) on the raw sample guided the preprocessing choices. As shown in Figure 2, the target `overall_rating` is roughly symmetric and bounded, so we modeled it directly without transformation.

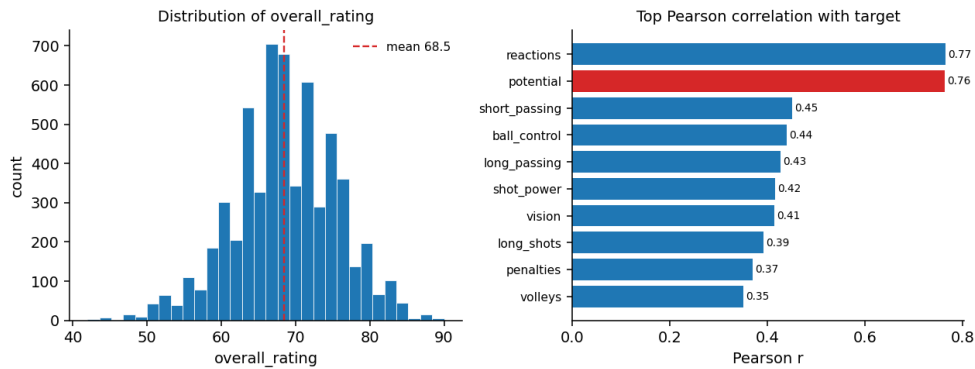


Figure 2: Left: Distribution of `overall_rating` (mean 68.5). Right: Top Pearson correlations with the target. Note `potential` is highly correlated but must be dropped to prevent target leakage.

The strongest predictor, `potential` ($\rho \approx 0.76$), was dropped because it would not be available at inference time in a real-world scouting scenario. For the work-rate features, which contained noisy labels (e.g., `norm`, `stoc`), we mapped valid inputs to an ordinal scale ($\{0, 1, 2\}$) and imputed invalid entries with the mode (medium). Missing skill ratings were imputed with the column median, which we saved for use during serving. We also added age, BMI, an `is_goalkeeper` flag, and average ratings by FIFA skill category.

The `data_quality` pipeline validates the raw 6,000-row sample against 19 rules (schema, ranges, nulls), which all pass.

3.2 Modelling and validation

We evaluated four models using 5-fold cross-validation. Table 2 shows that the tree-based models achieved lower RMSE than the linear baselines, suggesting that the linear models did not capture the relationship between attributes and rating as well. We selected HistGradientBoosting as the champion model. On the held-out test set, it achieved an RMSE of 1.45, MAE of 1.05, and an R^2 of 0.954.

Model	CV RMSE (pts)
HistGradientBoosting (champion)	1.59
Random Forest	1.93
Linear Regression	3.29
Ridge	3.30

Table 2: Five-fold cross-validated RMSE across candidates (`model_selection_report.json`).

The high R^2 is expected because the features include the same types of skill attributes used to assign

FIFA ratings. However, because the dataset contains multiple snapshots per player, a standard random split can leak player identity between train and test sets. When we performed a grouped split on `player_api_id` to separate players across splits, performance changed only modestly (RMSE 1.54, R^2 0.950, 84.2% within 2 points), with zero overlapping players between train and test. This suggests that player overlap was not the only reason for the strong test result. The robustness report includes goalkeeper/outfield and rating-band error slices, but it does not include protected-attribute subgroup or fairness metrics.

Diagnostic plots (Figure 3) show that predictions track observed ratings through most of the distribution, and the residual plot does not show a clear pattern.

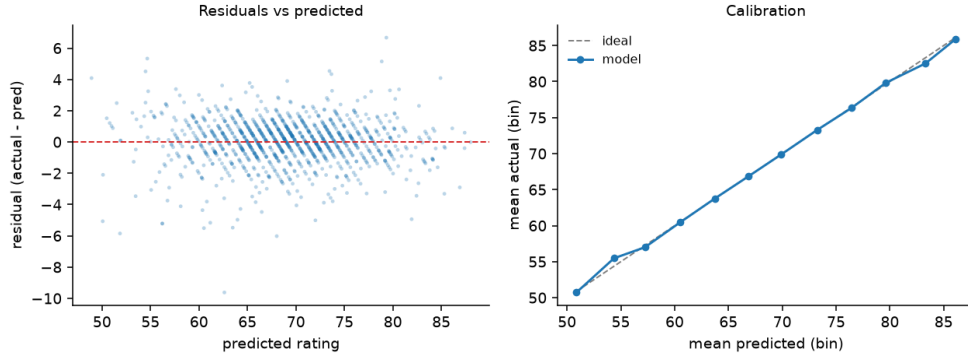


Figure 3: Left: Residual plot showing no distinct patterns. Right: Binned calibration plot tracking the identity line.

For reproducibility, running the pipeline twice in a clean environment yielded the exact same champion model and test metrics.

3.3 Explainability (SHAP)

The `model_train` node generates feature importance metrics using SHAP (falling back to permutation importance if necessary). Figure 4 shows the features with the largest contribution to predictions.

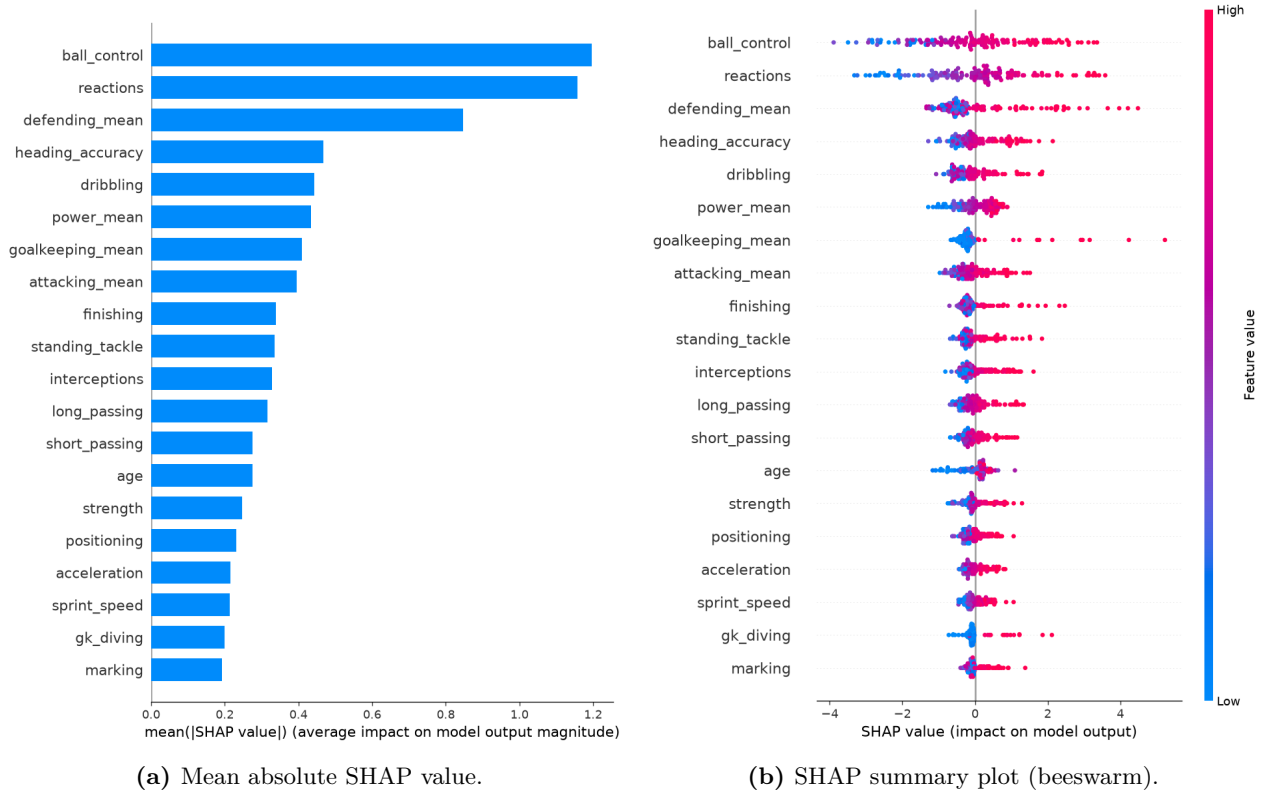


Figure 4: Feature explainability. `ball_control` and `reactions` are the primary drivers of predictions.

The SHAP summary plot shows that `reactions` and `ball_control` have the largest contributions, consistent with their high correlations in the initial EDA. The `goalkeeping_mean` feature helps separate goalkeeper profiles from outfield-player profiles, allowing one model to handle both groups.

3.4 Drift monitoring

To detect distribution shifts, the `data_drifts` node compares new batches to the training distribution using Population Stability Index (PSI) and the 2-sample Kolmogorov-Smirnov (KS) test.

The baseline run (`drift_report.json`) flagged 1 out of 48 features. With $\alpha = 0.05$ across 48 feature tests, one flagged feature is not enough by itself to treat the batch as drifted, so no dataset-level drift was triggered. As a smoke test for the monitoring logic, we simulated a drift scenario (`drift_report_simulated.json`), which flagged 18 features and raised a dataset-level alert. Figure 5 compares the PSI scores between the clean and drifted batches. The current implementation monitors feature drift only. Target drift and concept drift would require later batches with observed outcomes.

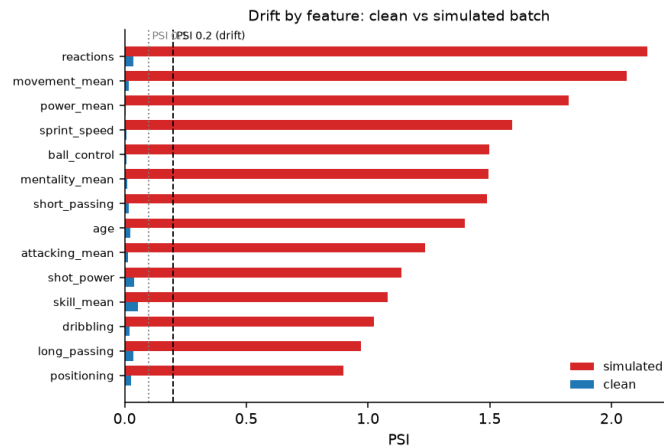


Figure 5: Feature-level PSI for the clean batch (blue) and the simulated drift batch (red). The simulated batch crosses the 0.2 PSI threshold for multiple features, triggering a global drift alert.

4 Future Work & Production Roadmap

The current pipeline runs locally on a static CSV sample. Moving it toward production would mainly require changes in orchestration, shared tracking, data validation, storage, and alerting.

Next (Immediate upgrades):

- **Orchestration & Scheduling:** The Kedro CLI is useful for local runs, but scheduled execution should be handled by an orchestrator. We plan to deploy this via `kedro-airflow` so we have proper scheduling and retries.
- **Tracking Server:** Our MLflow setup currently uses a local SQLite file. We need to migrate this to a hosted MLflow server with an object-store backend (like S3) so runs, artifacts, and registered models are shared across the team.
- **Data Quality:** Replace the manual `assert` checks with a Great Expectations suite running in CI.

Later (Scaling up):

- **Distributed Compute:** Pandas is sufficient for the current 184k-row table, but larger datasets may require porting the pipeline nodes to Spark or Dask.
- **Online Feature Store:** Local Parquet files work for offline training. For real-time serving, we'll need to adopt a feature store like Feast or Hopsworks to handle point-in-time joins and serve features online.
- **Drift Alerting:** The Evidently reports are batch-only right now. We need to set up automated alerting and a retrain trigger when the KS/PSI thresholds are breached.

If a clean, independently sourced version of the `potential` feature becomes available, it should be added through the feature store and treated as a new model version, with leakage checks and retraining before use.

5 Environment & Dependencies

The environment is declared in `pyproject.toml`. Rather than listing every transitive package, here are the main packages used by the pipeline:

- **Pipeline & Data:** `kedro` (0.19.x) for orchestrating the DAG, with `pandas`, `numpy`, and `pyarrow` for data wrangling.
- **Modelling & Metrics:** `scikit-learn` (1.x) for candidate models and preprocessing, with `scipy` (1.1x) backing the statistical drift tests.

- **Tracking & Serving:** `mlflow` (3.x) handles the model registry, while `fastapi` and `uvicorn` run the REST API.
- **Monitoring & Explainability:** `evidently` (0.7.x) generates the HTML drift reports, and `shap` (0.5.x) provides feature importance metrics.
- **Testing:** `pytest` runs our 30 unit and integration tests.